

Threat Analysis (Kubernetes secrets vs. Spiffe/Spire)

1 Prerequisites.....	2
1.1 Scope.....	2
2 Kubernetes Secrets.....	3
2.1 Discussion.....	5
3 Spiffe/Spire Workload Identity.....	6
3.1 Discussion.....	8
4 Comparison Summary.....	10

1 Prerequisites

In order to understand the models, several aspects need to be stated. Here, we explain the scope and other aspects that will clarify what the model does, and what it does not do. The chosen strategy for this comparative model is to make one model for each scenario where we focus on the actual differences between the two. As such, the model is not a complete model. It is highly tailored to the specific scopes.

In general, the model does not aim to be as granular as possible. Rather, it aims to be informative and digestible for anyone without prior knowledge of these systems.

Another thing to mention is that from our research, people recommend various approaches to performing STRIDE. While there does exist convention on the process in general, the intricacies differ to large degrees. For example, we found one document explaining that when marking the STRIDE aspects on a component, one should consider what could be done to this component, while another explained that one should consider what could be done if one has gained access to this component. Our choice is the latter, meaning that our listed STRIDE aspects describe what can be done if an adversary has access to the component.

1.1 Scope

This is a comparative threat analysis between Kubernetes secrets and Spiffe/Spire. Specifically, it focuses on the perspective of workloads that use either technique for communication. This results in a few observations that might seem odd at first. For example, the Spiffe/Spire model does not have the etcd component, while the model for secrets does. While etcd still exists in that scenario, and might be used to store secrets for other parts of Kubernetes, it is practically non-existent from the perspective of workloads.

Another aspect to note is that some threats might exist that are the same regardless of which model is used. We do not include these in our model. Again, the focus is on the difference between the two.

Lastly, it also draws boundaries at some points. For example, the storage of secrets for a workload can be as environmental variables. This puts the storage in the operating system ultimately, which we consider to be worth mentioning, but also outside the scope of the model. If it were to be included, we would have to model attacks concerning an operating system which would make the entire model less digestible.

2 Kubernetes Secrets

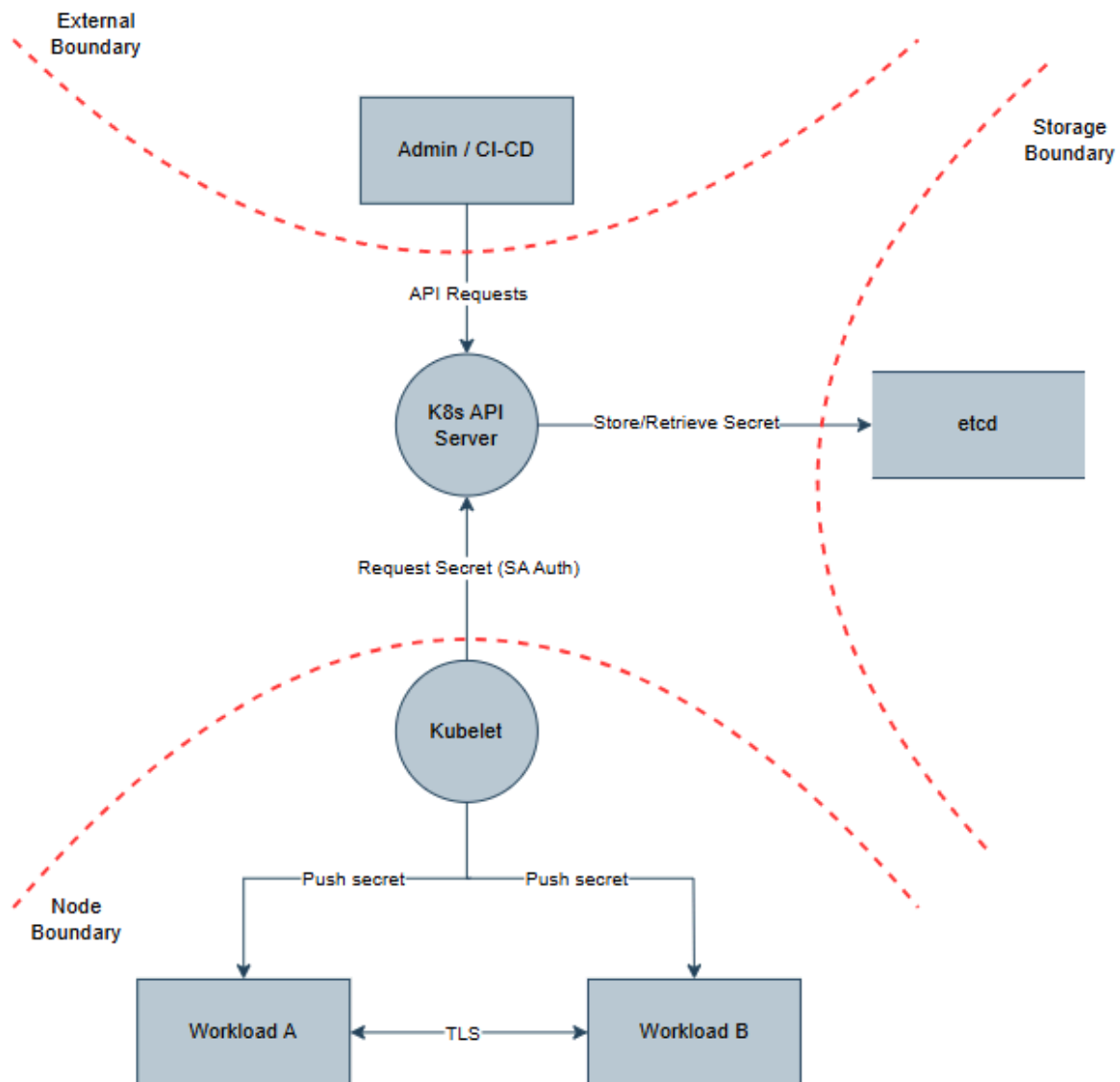


Figure 1 - Kubernetes Secrets DFD

Kubernetes secrets are stored in the etcd, which is a storage for base64-encoded key value pairs. When a workload starts, Kubelet (who is responsible for runtime) requests the secret from the API server; the API server then fetches the secret from etcd and delivers it to Kubelet who subsequently grants it to the workload. This can either be stored as a mounted volume, or as environmental variables. A volume would be modeled as an extra component, local to the workload. If stored as environmental variables, it is stored in the process memory of the workload, which makes it part of the operating system. Most important here is understanding of the flow described between the workload and the control plane (API server and etcd).

The boundaries seen in the model are the trust boundaries. In this case, they describe differences in needed credentials. For example, the API server does not have automatic access to the etcd, it actually needs to authenticate itself. The same goes for the kubelet which needs to authenticate to the API server. Lastly, the same goes for the admin / CI-CD pipeline, which might be an automated process, or an externally located user/admin.

	Admin / CI-CD
S: Credentials can be stolen in order to access the API server and subsequently the etcd, where secrets are stored	
R: If logging is not properly configured, repudiation is a risk	

	K8s API Server
T: Using API commands to alter, delete, or add secrets to etcd	
R: If logging is not properly configured, repudiation is a risk	

	etcd
T: Stored secrets can be directly altered, deleted or added	
R: If logging is not properly configured, repudiation is a risk	
I: Secrets can be exposed and read	
D: Deleting or corrupting stored secrets can affect availability	

	Kubelet
R: If logging is not properly configured, repudiation is a risk	
I: Secrets can be exposed and read	
D: Deleting or corrupting stored secrets can affect availability	

	Workload
T: Secrets can be directly altered, deleted or added	
I: Secrets can be exposed and read	
R: If logging is not properly configured, repudiation is a risk	
D: A compromised workload can potentially crash the workload	
E: Shell can potentially be escaped, granting access to the entire node	

Figure 2 - Kubernetes Secrets STRIDE

Figure 2 contains the STRIDE analysis done on each component. Further discussion is left to the next section.

2.1 Discussion

In general, there are several ways in which the environment in the model can be attacked. The Admin / CI-CD is a path that could grant someone access to the API server. Once an attacker has access to the API server, they can modify or expose secrets from the etcd. These secrets can then be re-used. We may also note that having access to the control plane can be disastrous for the whole environment, but that is outside the scope of “workload communication”.

The kubelet and the workloads sit on the same node in the cluster. If a kubelet is compromised, it may act as a man in the middle when requesting secrets from the control plane. Another possibility is that if the roles for the kubelet are too permissive, it could ask the API server to modify its own privileges, effectively granting it access to the whole cluster. This would mean that this is another point of which etcd could be compromised.

As seen between Workload A and Workload B, there is a communication arrow with a tag saying “TLS”. This is of course not the only possible protocol, but it is a common one, and it is mainly noted here to illustrate the difference when mTLS can be used with Spiffe/Spire. The difference between the two is that on TLS, the client verifies the server, but not vice-versa. With mTLS, they both verify each other. We will revisit this in the summary.

Key Takeaways

There are several attack paths in this scenario and most of them can potentially gain access to secrets. The main problem with secrets is that they are long-lived (they do not rotate/change) and can be used by other entities.

3 Spiffe/Spire Workload Identity

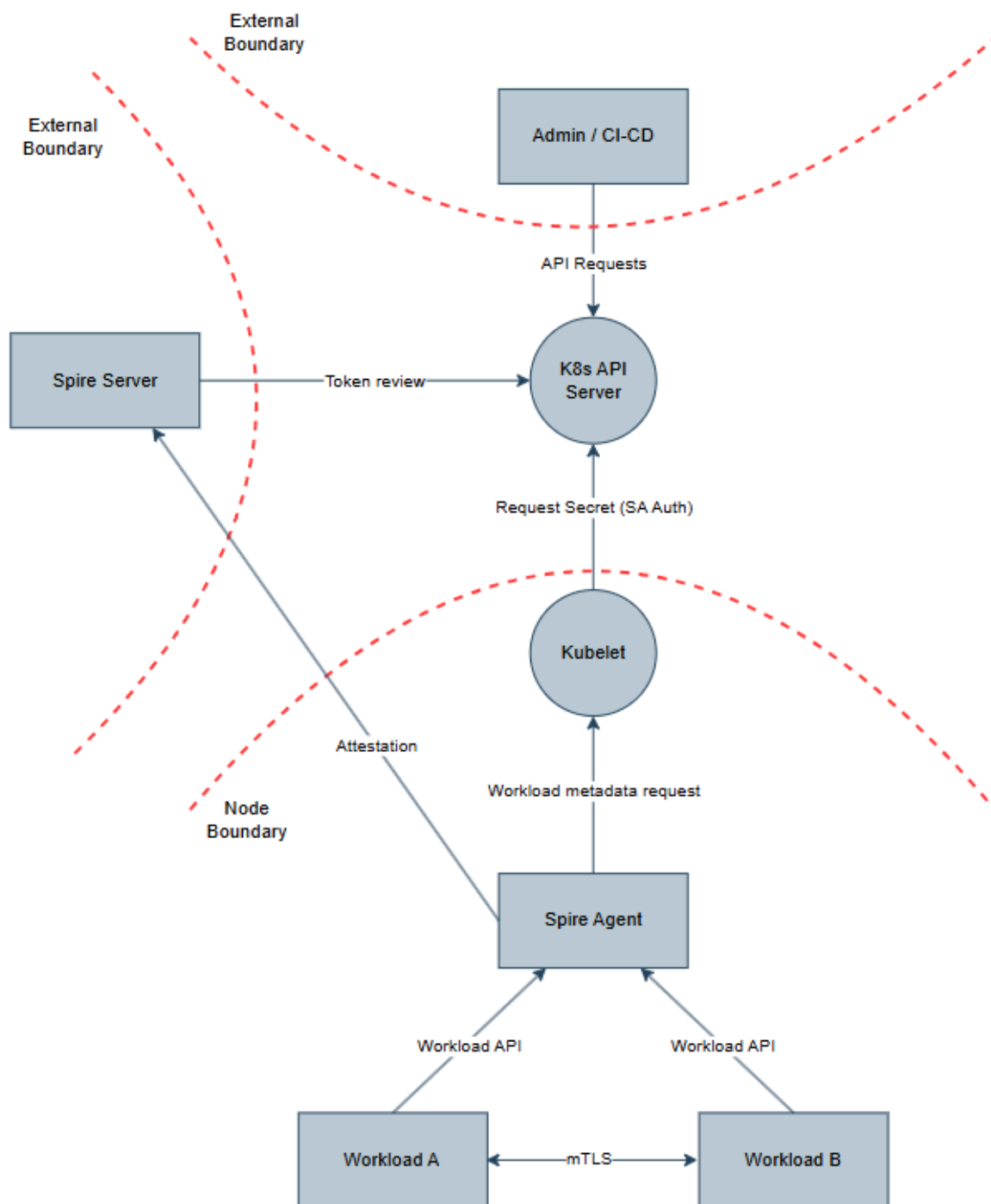


Figure 3 - Spiffe/Spire DFD

The spire server is configured with a list of “allowed” entities. For example, it may only allow Workloads from node “x”, with service account “y”, under the agent “z”. It also acts as a certificate authority. It builds a trust chain and the root of this trust is the spire server. However, this trust is also dependent on the Kubernetes cluster. The relationship to be described will explain why.

A spire agent waking up on a node will attempt to attest to the spire server. When it wakes up, the Kubernetes control plane will grant it a PSAT (Projected Service Account Token). This is a signed JWT-token (Json Web Token) containing information on who the agent is and where it lives. It sends this information to the Spire server who validates its properties to ensure it is an allowed entity. It then sends the PSAT to the Kubernetes API server to ask for validation (integrity of the token). Assuming everything is in order, it grants the Spire agent an x.509 certificate with its Spiffe ID. By default, this is valid for one hour and is automatically rotated by the Spire server.

At this point, the identity of that Spire agent has been confirmed. Enforced and granted by the Spire server, essentially vouched for by the Kubernetes control plane. This is why the Kubernetes cluster is also considered a root of trust. The Spire server now confidently knows that the agent is not an adversarial entity, because Kubernetes has confirmed that it lives within its cluster, and has certain properties such as a specific service account, and more.

This is where the chain becomes a chain. The Spire agent can now act on behalf of the Spire server. From the server, it gains the trust bundle and a list of allowed entities that it stores to cache, it also stores certificates for these entities that it can grant on request, or when rotation is needed.

There are various routines for which a workload can be given its x.509 SVID, we describe one of them here. A workload spins up at the same node as its agent, it immediately contacts the spire agent to attest in order to get its certificate. The agent not only checks its list of allowed entities, but also checks the workloads service account and other Kubernetes properties. Furthermore, it also “interrogates” the operating system by checking metadata of the workload process. This, again, is to ensure that the entity contacting the workload is who it claims to be, and not some other device living outside the cluster. Assuming everything is in order, the agent grants the workload its certificate, which can now be used to communicate via mTLS with other workloads, not only inside the cluster, but any workload within the Spiffe domain.

As mentioned in the prerequisites, etcd is not included in this model simply because it is not relevant for workloads in this scenario. It still very much exists, and can do other harms to the cluster.

Admin / CI-CD
S: Credentials can be stolen in order to access the API server and subsequently the etcd, where secrets are stored
R: If logging is not properly configured, repudiation is a risk

K8s API Server
T: Using API commands to alter, delete, or add workload registration entries
R: If logging is not properly configured, repudiation is a risk

Kubelet
R: If logging is not properly configured, repudiation is a risk

Workload
R: If logging is not properly configured, repudiation is a risk
D: A compromised workload can potentially crash the workload
E: Shell can potentially be escaped, granting access to the entire node

SPIRE Server
S: Credentials can be stolen in order to access the SPIRE server.
T: Using API commands to alter, delete, or add workload registration entries
D: Deleting or corrupting the SPIRE Server can prevent SVIDs from being issued

SPIRE Agent
S: A compromised node can be used to impersonate the SPIRE Agent to obtain unauthorized SVIDs
T: The agent Unix socket can be tampered with to intercept or manipulate SVID delivery
R: If logging is not properly configured, repudiation is a risk

Figure 4 - Spiffe/Spire STRIDE

3.1 Discussion

Just as in the case of Kubernetes Secrets, Admin / CI-CD can be used by adversaries to gain access to the cluster. In this scenario however, the secrets regarding workloads cannot be stolen, because no secrets are used for workloads. While many attack paths remain in general, and other general threats also remain, this perspective of workloads dramatically reduces the paths and threats related to workload communication. Those that remain, might be harder to perform since configuring the agent to bypass its own checks for example, would be more complex than to send a request to the etcd for credentials, which is a standard flow.

If an adversary gains access to the control plane via Admin / CI-CD, it could interfere with the validation process of a Spire agent's PSAT validation. It would essentially have configured the PSAT service of the API server to return "it matched" to the spire server. This is possibly the remaining risk that has the highest impact potential.

If the Spire agent gets compromised, it could be able to grant certificates to other than intended entities. Due to its own certificate being rotated, this would only last for a maximum of one hour, unless it can repudiate effectively.

If the Kubelet gets compromised, it can no longer spy or interrupt the flow of secrets, because it is not part of this aspect any longer. It can potentially gain access to certificates and SVIDs, but these are essentially useless since they are being rotated frequently. This is also true for workloads.

One thing to note is that if a certificate gets stolen, it will work for the full hour on other entities unless it can be detected and revoked. We should also note that one hour is the default, it can be changed to more frequent rotations.

The Spire server becomes an additional attack path in this scenario. As it acts as the CA, it is a high-value target.

Key Takeaways

This scenario uses certificates, where the spire server acts as the CA. Certificates automatically rotate, which limits the window of opportunity for adversaries. It does however, introduce a new high-value target, the Spire Server.

4 Comparison Summary

From the perspective of workloads and stolen credentials (secrets/certificate), Spiffe, through the Spire implementation, seems to deliver an increase in overall security. While the attack surface to an extent remains, stealing or spying on a certificate (through manipulating the kubelet, the agent, the spire server, or the control plane) is more difficult to perform than to fetch the secrets written in plain text within etcd.

The biggest increase in security comes from abandoning long-lived secrets for short-lived certificates. If a secret is stolen, it can be reused by any entity, and in order to stop this, one would have to manually alter any other entity using the same shared secret, and the servers who accept them. With short-lived certificates, they rotate automatically and can practically not be used anywhere else. Even if one were stolen, they would be either revoked or useless by the time of rotation. This significantly reduces what is known as the blast-radius, i.e. how many systems/devices can be affected by a stolen credential.

Spiffe does add a new attack path, the Spire server. While it is high-value, its status as a CA means it will have to be well protected. Other than this, they are both vulnerable to control of the Kubernetes cluster.

Lastly, the use of certificates allows the use of mTLS for communication. As mentioned, this is the difference between only having a client verify the server (TLS) or by having them both verify each other. This is the main reason why a window of opportunity stays only a window. If an adversary manages to get a valid certificate, it will be rejected by the server once the certificate is supposed to be rotated.